JFEM applications

Installation and local sysimages

Windows, macOS and Linux

One checkout. One setup for JFEM, FGen, PANDEATOR and the browser results viewer.

Start with Julia and the complete public checkout. One command installs two package environments and verifies all four entry points. Then launch each tool, check a bundled model, and optionally build, use and rebuild a local solver sysimage.

What you will install

Component	Purpose
Julia	Runtime and package manager.
JFEM	Finite element solver, in the repository root.
FGen / WingFEGen	Local browser application for geometry, FEM generation, analysis and results.
PANDEATOR	Stiffened-panel case runner; uses the root solver project.
Results viewer	Static browser viewer for existing `.jfem` files; no Julia server.
Julia packages	Installed automatically from each project's recorded dependency files.

The complete repository is self-contained with respect to project source and browser assets. Normal use needs no private development folders, commercial reference solver, Python, Node.js or separately installed web server. Internet access is required to obtain Julia, the repository and missing Julia packages/artifacts.

Runtime and storage

This checkout is tested with **Julia 1.13.1** and records it in both committed manifests. Install the matching release line from the official Julia distribution. Later patches in the 1.13 release line are accepted. A newer Julia minor release should be used with a matching future checkout; setup does not silently rewrite this checkout's pinned environment.

Choose a writable folder, for example `C:\Work\JFEM` or `~/JFEM`. Julia packages normally occupy the user depot `~/.julia`. Downloads and first compilation can take several minutes. Normal setup does not build a sysimage or run a user analysis.

Route through this guide

Windows: page 2. macOS/Linux: page 3. Checks: page 4. PANDEATOR and viewer: page 5. Sysimage build/use: pages 6–8. Repair and updates: page 9.

Commands assume the terminal is in the repository root unless stated otherwise. Keep all browser assets and shared folders in the complete checkout.

Windows: install, download, set up

1. Install Julia

Open PowerShell and run Julia's official Juliaup installation command:

```
winget install --name Julia --id 9NJNWW8PVKMN -e -s msstore
```

If the Microsoft Store is blocked, follow the alternative installers on the official Julia installation page. Close and reopen PowerShell after installation, then run:

```
julia --version
```

The current checkout expects the Julia 1.13 release line. If the default Julia is from another release line, install a matching runtime alongside it:

```
juliaup add 1.13
julia +1.13 --version
```

This does not require changing your Juliaup default.

2. Get the entire repository

With Git installed:

```
git clone https://github.com/r-acad/JF_2026_05_25.git JFEM
cd JFEM
```

Alternatively, open the GitHub repository, choose **Code > Download ZIP**, extract the complete archive into a writable folder and open PowerShell there. Do not launch files from inside the ZIP. The extracted directory may have another name; its root must contain `setup.cmd`, `Project.toml`, `Manifest.toml`, `src`, `JFEM_installation` and `WingFEGen`.

3. Set up all applications automatically

```
.\setup.cmd
```

Setup installs the root and FGen environments, precompiles packages, and verifies JFEM, FGen, PANDEATOR and the static-viewer assets. Wait for the completion summary. No manual package list is needed. To select Julia explicitly:

```
julia +1.13 --startup-file=no JFEM_installation/julia_tools/install_julia_packages.jl
```

4. Open FGen

```
.\WingFEGen\run_fe_generator.cmd
```

Keep the terminal open. The browser opens the local application after startup. The default address is http://127.0.0.1:8080. Use the address printed in the terminal if a different port was selected.

macOS and Linux: install, download, set up

1. Install Julia

Open a terminal and run Julia's official Juliaup installer:

```
curl -fsSL https://install.julialang.org | sh
```

Follow the installer prompts. Close and reopen the terminal, then check:

```
julia --version
```

Prefer the official Julia distribution. If this checkout's recorded 1.13 release line is not installed, add it and select it explicitly:

```
juliaup add 1.13
julia +1.13 --version
```

Juliaup can update the latest stable channel with `juliaup update release`. Keep a compatible runtime available for each recorded project environment.

2. Get the repository

```
git clone https://github.com/r-acad/JF_2026_05_25.git JFEM
cd JFEM
```

Git is optional: the GitHub **Code > Download ZIP** route also works. Extract all files before continuing. Keep `WingFEGen` within the solver checkout so the application finds JFEM automatically.

3. Set up all applications

```
bash setup.sh
```

Invoking Bash directly avoids depending on an archive's executable-bit preservation. The equivalent portable command, including explicit runtime selection if needed, is:

```
julia +1.13 --startup-file=no JFEM_installation/julia_tools/install_julia_packages.jl
```

Installation may download native binary artifacts as well as Julia source packages. Wait for the completion message. Setup changes neither your global Julia project nor your startup file or persistent depot configuration.

4. Start the local application

```
julia --startup-file=no --project=WingFEGen WingFEGen/run.jl
```

To select a port and open the browser yourself:

```
julia --startup-file=no --project=WingFEGen WingFEGen/run.jl --port 9000 --no-browser
```

Use the printed local URL. Stop the application with Ctrl+C in the terminal after saving your work.

Verify the installation and begin work

What successful setup looks like

The console names both projects, installs their recorded environments and reports precompilation. Verification covers JFEM and FGen module loading, PANDEATOR's server/health entry point and the static viewer's local assets. PANDEATOR verification starts no user server and performs no warm-up solve. Wait for the completion summary.

The normal launch checks package source availability. If packages are absent, it reports and installs them automatically before importing the application. Ready environments take the fast path without refreshing registries. Explicit setup should be rerun to repair missing artifacts after an interrupted download.

Run the bundled tiny solver example

No external input is required. In Windows PowerShell:

```
$deck = "JFEM_installation\examples\precompile\sol101_quad_static.bdf"
.\jfem.cmd "$deck" output\installation_smoke
```

In macOS/Linux:

```
deck="JFEM_installation/examples/precompile/sol101_quad_static.bdf"
bash jfem "$deck" output/installation_smoke
```

A successful analysis writes `.jfem` viewer data, a report and results JSON under `output/installation_smoke`. Adding `-rs` before the deck selects only report and JSON, omitting viewer binary data. The first analysis may compile additional methods.

Check FGen without opening a server

```
julia --startup-file=no --project=WingFEGen WingFEGen/run.jl --help
```

For normal use, start FGen as on the previous pages, choose **Start new study** or open an existing study, generate the FEM and choose an analysis. The solver is discovered from the containing checkout. Save studies to a writable folder and retain your results when updating the software. The separate FGen user guide describes modelling and result inspection.

Preparation options

Setup option	Effect
`--no-precompile --no-verify`	Download/install only. Future imports still compile. This does not verify successful module loading.
`--solver-only`	Prepare the root project and associated tools; omit the FGen project.
`--wingfegen-only`	Prepare FGen only; not a complete solver-backed installation.
`--offline`	Use a previously populated matching depot; cannot bootstrap an empty offline machine.
`--dry-run`	Print and validate the project plan without package installation.

Add `--sysimage` to build the optional solver/PANDEATOR image after setup. Neither `--no-precompile` nor `--no-verify` cancels that explicit build. Normal operation does not require an image; details follow.

PANDEATOR and the browser results viewer

PANDEATOR: generate and solve stiffened-panel cases

PANDEATOR uses the root Julia environment, so the unified setup already prepares it. In Windows PowerShell:

```
.\POST\PANDEATOR_APP\RUN_PANDEATOR_WINDOWS.cmd
```

On macOS/Linux:

```
bash POST/PANDEATOR_APP/RUN_PANDEATOR_MAC_LINUX.sh
```

The default address is http://127.0.0.1:8088/. Wait for readiness; ordinary startup can warm representative solver paths before the server listens. Keep the terminal open. Both wrappers accept:

```
--port 8090 --no-open
```

Use those arguments to select another port and open the browser yourself. FGen's corresponding switch is `--no-browser`, not `--no-open`. Stop your server with Ctrl+C after saving its outputs.

Static viewer: open existing solver results

Open this local file in a modern browser:

```
POST/JFEM_results_viewer/postv11.html
```

Choose or drop a `.jfem` file, such as the output of the bundled check on page 4. The static viewer needs neither a Julia server nor separate packages. It cannot launch a solve. Keep the full `POST` tree: the HTML loads local Babylon and the shared binary reader by relative paths. A report/JSON-only export does not supply the required binary result.

Which application should I open?

Task	Entry point
Lifting-surface model or imported deck	FGen; its local server normally uses port 8080.
Stiffened-panel case runner	PANDEATOR; its local server normally uses port 8088.
Batch/command-line FE analysis	`jfem.cmd` on Windows or `bash jfem` on macOS/Linux.
Inspect existing binary results	Open `postv11.html`; no server port.

Normal solver/FGen/PANDEATOR launch paths can install missing package sources before importing modules. Explicit setup is the clearest all-app preparation and repair step. `JFEM_AUTO_INSTALL=0` disables automatic installation in managed/offline workflows.

More help. Read `POST/PANDEATOR_APP/PANEL_APP_README.md` and `POST/JFEM_results_viewer/POST_GUIDE.html`. The illustrated FGen user guide is alongside this PDF in `Reference_documentation/00_Getting_Started/`.

Optional sysimage: build it locally

A sysimage stores loaded packages and selected compiled methods to reduce repeated startup and first-use compilation. It does not replace Julia or the checkout, and methods absent from the workload may still compile later. Embedded package versions are fixed at build time.

Scope. This repository's image contains OpenJFEM and the PANDEATOR server stack/workload. Eligible native solver processes launched by FGen can use it. It does *not* compile the separate FGen application server or all FGen geometry/UI paths. The static viewer uses no Julia image.

Build with the unified command

Stop your own solver/server processes before replacing an image they may have loaded. Keep source and representative input decks unchanged during the build. Use the same compatible Julia executable that will run the applications.

```
# Windows PowerShell
.\setup.cmd --sysimage

# macOS/Linux
bash setup.sh --sysimage
```

The portable equivalent is:

```
julia --startup-file=no JFEM_installation/julia_tools/install_julia_packages.jl --sysimage
```

Wait for dependency setup, workload compilation, native image creation and the provenance receipt. The build executes small representative analyses. It can take several minutes and substantial CPU/RAM. Read workload warnings; creation of a file alone does not prove complete coverage.

Platform	Standard image under the checkout
Windows	`sysimage/OpenJFEM_sysimage.dll`
Linux	`sysimage/OpenJFEM_sysimage.so`
macOS	`sysimage/OpenJFEM_sysimage.dylib`

Each image has an adjacent receipt with `.jfem.toml` appended to its full name, for example `OpenJFEM_sysimage.dll.jfem.toml`. Retain both files.

Compiler prerequisites and existing click scripts

PackageCompiler needs a C compiler. It obtains one automatically on Windows when required. macOS can use clang from Xcode Command Line Tools; Linux can use a current gcc or clang. Prefer official Julia binaries. See the PackageCompiler prerequisites.

These existing files under `JFEM_installation` delegate to the same setup with `--sysimage`:

```
CLICK_WINDOWS_INSTALL_PACKAGES_AND_CREATE_SYSIMAGE.cmd
CLICK_MAC_INSTALL_PACKAGES_AND_CREATE_SYSIMAGE.command
RUN_LINUX_INSTALL_PACKAGES_AND_CREATE_SYSIMAGE.sh
```

Double-click the platform file or invoke the shell script with Bash. Workload arguments are forwarded.

Sysimage: verify, use and rebuild

1. Check its provenance receipt

From the root, on Windows:

```
julia --startup-file=no --project=. JFEM_installation/julia_tools/check_sysimage.jl .
    sysimage/OpenJFEM_sysimage.dll
```

Use `.so` on Linux or `.dylib` on macOS. Successful verification is silent with exit code zero. A nonzero exit code prints the rejection reason and announces normal package loading.

2. Use the ordinary launch commands

The `jfem.cmd`/`jfem` and PANDEATOR wrappers search the standard image path and check the receipt before loading it. FGen's eligible native solver runs check it too. The FGen application server remains in its own environment. Repeat the tiny solver run from page 4 to verify actual operation with the selected runtime. This does not validate every solver feature or a user's model.

The receipt compares the exact Julia/runtime/CPU identity, checkout path, source/project inputs, image hash and representative workload. A missing receipt, moved checkout, changed code/manifest/image or changed/missing workload deck triggers ordinary package loading. This is an eligibility check, not an audit of arbitrary depot edits or system libraries.

3. Rebuild when inputs change

Rebuild after a Julia upgrade, including a patch change; project/dependency or tracked code changes; a checkout move; a machine/OS/CPU change; or changes/removal of a representative workload deck. Stop your own affected processes, update the complete checkout and rerun:

```
.\setup.cmd --sysimage
# or
bash setup.sh --sysimage
```

Then check the new receipt and run the small solve again. Package setup can accept a newer Julia patch while an older image still needs rebuilding for that exact runtime. Do not copy another computer's image as a deployment shortcut.

4. Run without an image when diagnosing a problem

Start fresh Julia without `-J`/`--sysimage`:

```
julia --startup-file=no --project=. JFEM_installation/julia_tools/run_bdf.jl
    JFEM_installation/examples/precompile/sol101_quad_static.bdf output/source_smoke
```

Or archive the optional image and receipt together outside the standard location while no process uses them. The wrappers then load normal packages. Do not force a stale image with `-J`: direct Julia image loading bypasses the automatic receipt check and may load older compiled code.

Advanced sysimage workload and direct builder

Unified setup invokes the existing `JFEM_installation/julia_tools/deploy_fast.jl`. Without workload arguments, it uses tiny decks in `JFEM_installation/examples/precompile`. These warm common parser, assembly, solve and PANDEATOR paths; they cannot cover every future method specialization.

Choose representative inputs and build threads

```
julia --startup-file=no JFEM_installation/julia_tools/install_julia_packages.jl --sysimage --deck
    path/to/representative.bdf --sysimage-threads=4
```

Option	Meaning
`--deck PATH`	Add one representative deck; repeat for multiple inputs.
`--manifest PATH`	JSON with a `cases` array; each case has `input` or `deck`, resolved relative to the manifest.
`--flags VALUE`	Comma-separated `NAME=value` workload settings.
`--sysimage-threads=4`	Build-child Julia thread count; `auto` selects automatically. It is not a memory cap.
`--sysimage=PATH`	Explicit output image path; automatic launchers still look at the standard location.
`--dry-run`	Print the complete plan without installing/building.

Workload options require `--sysimage`. An image can be combined with `--solver-only`, but not with `--wingfegen-only`. Keep workload files at stable paths; the receipt records their locations and hashes.

The original direct build command

After unified setup, experienced users can invoke the existing builder directly. Windows:

```
julia --threads=auto --startup-file=no --project=. JFEM_installation/julia_tools/deploy_fast.jl
    --sysimage=sysimage/OpenJFEM_sysimage.dll --install-packagecompiler
```

Use `.so` on Linux or `.dylib` on macOS. The current project already declares PackageCompiler. The final flag permits adding it in an older project where absent, which can change dependency files. Direct `deploy_fast.jl` is not all-app setup; without `--sysimage=...` it only refreshes package/workload precompilation.

A custom image path

Prefer the standard image path for the supplied launchers. For deliberate direct use, run the receipt checker from page 7 with the custom image path and require a zero exit code. Then pass `--sysimage=PATH` to the intended Julia process with the root project. Do not attach the solver image indiscriminately to the separate FGen server project.

PackageCompiler's sysimage documentation explains version locking and compiled-method coverage. Reducing startup latency does not change the FE problem, numerical tolerances or result accuracy.

Repair, updates and offline use

Symptom	Action
Julia not found	Reopen the terminal and check `julia --version`. Select an executable with `JFEM_JULIA` for setup/solver or `WINGFEGEN_JULIA` for FGen. These variables take an executable path, not extra flags.
Wrong Julia release	Run `juliaup add 1.13`; use the explicit `julia +1.13` command. FGen also searches compatible installed runtimes. Do not edit manifests to bypass compatibility.
Download or artifact error	Check network/proxy access, certificate configuration and writable folders. Rerun setup; downloaded content is reused. Retain the original error text.
Access denied	Extract the checkout into a writable user folder, outside protected system directories.
Browser cannot connect	Wait for the ready/listening message, keep Julia running and use its printed URL. Try `--port 9000` if 8080 is occupied.
Long first start	Read the reported stages. Downloads, package precompilation and first-use compilation can take several minutes. No user analysis runs merely because setup is compiling packages.

Update safely

Close active solver runs. With Git, update the complete checkout using `git pull`; with ZIP downloads, extract the new version into a separate folder. Retain saved studies and result directories. Run setup again to install the updated recorded environments. Rebuild an optional sysimage when its source/runtime provenance no longer matches.

Offline and managed installations

Populate the package/artifact cache on a connected machine with matching Julia release, operating system and architecture, then transfer that cache deliberately. Native artifacts are platform-specific. Run setup with `--offline` on the target. An empty depot cannot install without package sources and artifacts. The repository's offline notes provide more context; both current manifests are authoritative rather than historical package tables.

A temporary `JULIA_DEPOT_PATH` can select an isolated cache. Setup does not set it permanently. Managed workflows may set `JFEM_AUTO_INSTALL=0`; first launch will then explain the required setup command instead of downloading missing sources.

Official sources and maintained instructions

- Julia installation, including Windows alternatives:
  https://docs.julialang.org/en/v1/manual/installation/
- Current official Julia releases:
  https://julialang.org/downloads/manual-downloads/
- Julia project environments and recorded dependency installation:
  https://pkgdocs.julialang.org/v1/environments/
- JFEM source and downloadable ZIP:
  https://github.com/r-acad/JF_2026_05_25
- Maintained repository guide: `JFEM_installation/INSTALLATION.md`.